

CEFUSA E-Commerce

Explicación Técnica del Código Fuente

Arquitecturas de Software 2026-1

Entregable No. 1 — Núcleo de Negocio y API REST

5 de marzo de 2026

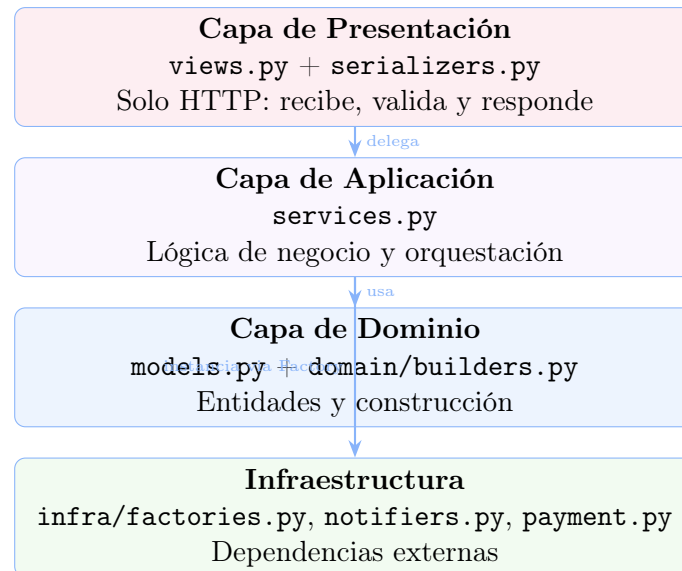
Documento autogenerado a partir del código fuente del proyecto

Índice

1. Visión General: La Arquitectura en Capas	2
2. Capa de Dominio: los Modelos	2
2.1. Product y ProductVariant	2
2.2. Inventory	3
2.3. Customer	4
2.4. Order y OrderItem	4
3. Patrón Builder: Construcción de Objetos Complejos	6
3.1. OrderBuilder	6
3.2. ProductBuilder	8
4. Patrón Factory: Creación de Dependencias Externas	9
4.1. El árbol de clases completo	9
4.2. Nivel 1 — Las Clases Base (el contrato)	9
4.3. Nivel 2 — Las Implementaciones (Mock vs Real)	10
4.4. Nivel 3 — Las Factories (la decisión centralizada)	12
4.5. ¿Cómo usa OrderService las Factories?	12
5. Capa de Aplicación: los Services	14
5.1. OrderService	14
5.2. ProductService	16
5.3. CustomerService	17
6. Capa de Presentación: Views y Serializers	18
6.1. Serializers: validación de entrada/salida	18
6.2. Views: solo HTTP, sin lógica	19
7. Resumen de los Principios SOLID en el Proyecto	20
8. Flujo Completo de una Compra (Checkout)	20
9. Estructura de Carpetas y su Justificación	21
10.Endpoints Disponibles	22

Visión General: La Arquitectura en Capas

El proyecto está dividido en **cuatro capas** bien diferenciadas. Cada capa tiene una responsabilidad específica y nunca mezcla responsabilidades de otra.



Regla de oro del proyecto

Una vista nunca toca la base de datos directamente. Toda lectura o escritura pasa por un *Service*. Los *Services* usan *Builders* para construir objetos complejos y *Factories* para obtener dependencias externas.

Capa de Dominio: los Modelos

Aquí están las **6 entidades** del proyecto (el 60 % del dominio).

Product y ProductVariant

Listing 1: products/models.py – Producto y Variante

```

1 class Product(models.Model):
2     CATEGORY_CHOICES = [
3         ('clothes', 'Ropa'),
4         ('accessories', 'Accesorios'),
5         ('other', 'Otros')
6     ]
7     name = models.CharField(max_length=150)
8     description = models.TextField()
9     category = models.CharField(max_length=50, choices=
10         CATEGORY_CHOICES,
11         default='other')
12     is_active = models.BooleanField(default=True)
  
```

```
12 created_at = models.DateTimeField(auto_now_add=True)
```

¿Qué es choices?

El parámetro **choices** limita los valores aceptados en la base de datos a una lista predefinida. Django la valida automáticamente y genera etiquetas legibles (*Ropa*, *.Accesorios*) para la interfaz de administración.

Listing 2: products/models.py – ProductVariant

```
1 class ProductVariant(models.Model):
2     product = models.ForeignKey(Product, related_name='
        variants',
3                                     on_delete=models.CASCADE)
4     sku = models.CharField(max_length=100, unique=True)
5     size = models.CharField(max_length=20, null=True, blank
        =True)
6     color = models.CharField(max_length=30, null=True, blank
        =True)
7     price = models.DecimalField(max_digits=10,
        decimal_places=2,
8                                     validators=[
5         MinValueValidator(0)])
9     is_available = models.BooleanField(default=True)
```

¿Qué es ForeignKey?

Una **ForeignKey** crea una relación **muchos-a-uno**. Aquí, muchas variantes pueden pertenecer a un solo producto. **related_name='variants'** permite acceder desde un producto a sus variantes con **producto.variants.all()**. **on_delete=CASCADE** significa que si se borra el producto, se borran todas sus variantes automáticamente.

Inventory

Listing 3: products/models.py – Inventory con métodos de validación

```
1 class Inventory(models.Model):
2     product_variant = models.OneToOneField(ProductVariant,
3                                             related_name='inventory', on_delete=
4                                                 models.CASCADE)
5     available_quantity = models.PositiveIntegerField(default=0)
6     minimum_stock = models.PositiveIntegerField(default=5)
7     updated_at = models.DateTimeField(auto_now=True)
8
9     def has_stock(self, quantity: int) -> bool:
10         """Validacion de negocio: hay suficiente stock?"""
11         return self.available_quantity >= quantity
12
13     def is_below_minimum(self) -> bool:
14         """Alerta de reposicion: el stock esta bajo el minimo?"""
15         return self.available_quantity <= self.minimum_stock
```

¿Por qué Inventory es una entidad separada?

El Inventory está separado de ProductVariant intencionalmente. Esto sigue el principio de **Separación de Responsabilidades (SRP)**: la variante describe *qué es* el producto, mientras que el inventario describe *cuántos hay*. La separación facilita auditorías y permite que los dos evolucionen independientemente.

OneToOneField vs ForeignKey

OneToOneField es como ForeignKey pero garantiza que **un solo inventario** corresponde a **una sola variante**. Si la variante es la llave del cuarto, el inventario es el único baúl que puede tener esa llave.

Customer

Listing 4: customers/models.py – Cliente

```

1 class Customer(models.Model):
2     nombre      = models.CharField(max_length=100)
3     apellido    = models.CharField(max_length=100)
4     email       = models.EmailField(unique=True)
5     telefono    = models.CharField(max_length=20, blank=True)
6     direccion   = models.TextField(blank=True)
7     created_at  = models.DateTimeField(auto_now_add=True)
8
9     @property
10    def nombre_completo(self):
11        return f"{self.nombre} {self.apellido}"

```

¿Qué es @property?

Un @property permite acceder a customer.nombre_completo como si fuera un campo, cuando en realidad es un método calculado. No se almacena en la base de datos; se calcula cada vez que se accede.

Order y OrderItem

Listing 5: orders/models.py – Orden con estados y snapshots

```

1 class Order(models.Model):
2     STATUS_CHOICES = [
3         ('pending', 'Pendiente'),
4         ('confirmed', 'Confirmado'),
5         ('shipped', 'Enviado'),
6         ('delivered', 'Entregado'),
7         ('cancelled', 'Cancelado'),
8     ]
9     customer      = models.ForeignKey('customers.Customer',
10                                     on_delete=models.PROTECT, related_name='
                                     orders')

```

```

11 fecha_creacion = models.DateTimeField(auto_now_add=True)
12 status         = models.CharField(max_length=20, choices=
    STATUS_CHOICES,
13                                     default='pending')
14 direccion_envio = models.TextField()
15 discount_code   = models.CharField(max_length=50, null=True,
    blank=True)
16 subtotal        = models.DecimalField(max_digits=10,
    decimal_places=2, ...)
17 discount_amount = models.DecimalField(max_digits=10,
    decimal_places=2, ...)
18 total           = models.DecimalField(max_digits=10,
    decimal_places=2, ...)
19 shipping_status = models.CharField(max_length=20, default='
    pending', ...)
20 tracking_number = models.CharField(max_length=100, null=True,
    blank=True)

```

Listing 6: orders/models.py – OrderItem con snapshots del precio

```

1 class OrderItem(models.Model):
2     order = models.ForeignKey(Order, related_name='items',
3                               on_delete=models.CASCADE)
4     variant = models.ForeignKey('products.ProductVariant',
5                                on_delete=models.PROTECT,
6                                null=True, blank=True)
7     product_name = models.CharField(max_length=150) # snapshot del
    nombre
8     quantity = models.PositiveIntegerField()
9     price = models.DecimalField(max_digits=10,
    decimal_places=2, ...)
10
11     @property
12     def subtotal(self):
13         """Precio unitario x cantidad al momento de la compra."""
14         return self.price * self.quantity

```

¿Por qué guardar `product_name` y `price` en el item?

Si el precio de un producto cambia mañana, las órdenes pasadas deben mantener el precio que el cliente **realmente pagó**. Estos campos son *snapshots* (fotografías en el tiempo). El campo `variant` es la FK al producto real, pero `product_name` y `price` son copias inmutables del momento de la compra.

Patrón Builder: Construcción de Objetos Complejos

¿Qué problema resuelve el Builder?

Cuando crear un objeto requiere **muchos pasos**, validaciones intermedias y cálculos, un constructor `__init__` normal se vuelve inmanejable. El **Builder** separa la *construcción* del *resultado final*. Se configura paso a paso y solo al llamar `build()` se crea el objeto —ya validado y completo.

OrderBuilder

Listing 7: orders/domain/builders.py – OrderBuilder completo

```
1 class OrderBuilder:
2     DISCOUNT_RATE = Decimal("0.10") # 10 % plano
3
4     def __init__(self):
5         self._customer = None
6         self._items = []
7         self._direccion_envio = None
8         self._discount_code = None
9
10    # Metodo 1: asignar cliente
11    def for_customer(self, customer):
12        self._customer = customer
13        return self # <-- retorna self: permite encadenar metodos
14
15    # Metodo 2: agregar un item con validacion de stock
16    def add_item(self, variant, quantity: int):
17        if quantity <= 0:
18            raise ValueError("La cantidad debe ser mayor a 0")
19
20        inventory = variant.inventory
21        if not inventory.has_stock(quantity):
22            raise ValueError(
23                f"Stock insuficiente para '{variant.product.name}'"
24            )
25
26        self._items.append({
27            "variant": variant,
28            "product_name": variant.product.name, # snapshot
29            "quantity": quantity,
30            "price": Decimal(str(variant.price)), # snapshot
31        })
32        return self
33
34    # Metodo 3: asignar direccion de envio
35    def with_shipping_address(self, address: str):
36        self._direccion_envio = address
37        return self
```

```

38
39 # Metodo 4: aplicar descuento
40 def with_discount(self, discount_code: str = None):
41     self._discount_code = discount_code
42     return self
43
44 # Metodo final: construir y persistir la orden
45 def build(self) -> Order:
46     # Validaciones finales obligatorias
47     if not self._customer:
48         raise ValueError("Se requiere un Customer")
49     if not self._items:
50         raise ValueError("La orden debe tener al menos un item")
51     if not self._direccion_envio:
52         raise ValueError("Se requiere direccion de envio")
53
54     # Calculos monetarios
55     subtotal = sum(i["quantity"] * i["price"] for i in self._items)
56     discount_amount = (subtotal * self.DISCOUNT_RATE
57                       if self._discount_code else Decimal(
58                           "0.00"))
59     total = subtotal - discount_amount
60
61     # Persistir la orden
62     order = Order.objects.create(
63         customer=self._customer,
64         direccion_envio=self._direccion_envio,
65         discount_code=self._discount_code,
66         subtotal=subtotal,
67         discount_amount=discount_amount,
68         total=total,
69     )
70     # Persistir cada item
71     for item in self._items:
72         OrderItem.objects.create(
73             order=order,
74             variant=item["variant"],
75             product_name=item["product_name"],
76             quantity=item["quantity"],
77             price=item["price"],
78         )
79     return order

```

Fluent Interface (encadenamiento de métodos)

Cada método del Builder `return self` al final. Esto permite escribir:

```
orden = (OrderBuilder()
        .for_customer(cliente)
```

```
.with_shipping_address("Calle 123")
.with_discount("SAVE10")
.build()
```

El código es legible como una oración: “para el cliente, con esta dirección, con este descuento, construir”.

ProductBuilder

Listing 8: products/domain/builders.py – ProductBuilder

```
1 class ProductBuilder:
2     def __init__(self):
3         self._name = None
4         self._description = ''
5         self._category = 'other'
6         self._variants = []
7
8     def with_name(self, name: str):
9         if not name or not name.strip():
10             raise ValueError("El nombre no puede estar vacio")
11         self._name = name.strip()
12         return self
13
14     def add_variant(self, sku, price, initial_stock=0,
15                    size=None, color=None):
16         if not sku or not sku.strip():
17             raise ValueError("El SKU no puede estar vacio")
18         if price < 0:
19             raise ValueError("El precio no puede ser negativo")
20         # Validar unicidad del SKU dentro del mismo builder
21         if sku in [v['sku'] for v in self._variants]:
22             raise ValueError(f"El SKU '{sku}' ya existe")
23         self._variants.append({
24             "sku": sku, "price": price,
25             "initial_stock": initial_stock,
26             "size": size, "color": color
27         })
28         return self
29
30     def build(self):
31         if not self._name:
32             raise ValueError("El nombre es obligatorio")
33
34         product = Product.objects.create(
35             name=self._name,
36             description=self._description,
37             category=self._category
38         )
39         for v in self._variants:
40             variant = ProductVariant.objects.create(
```

```

41         product=product, sku=v["sku"],
42         price=v["price"], size=v.get("size"), color=v.get("
            color")
43     )
44     Inventory.objects.create(
45         product_variant=variant,
46         available_quantity=v["initial_stock"]
47     )
48     return product

```

Patrón Factory: Creación de Dependencias Externas

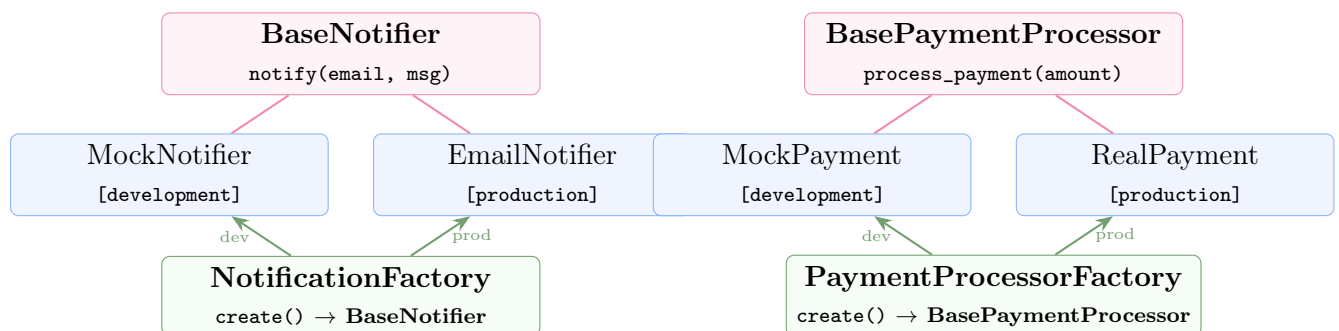
¿Qué problema resuelve el Factory?

El sistema necesita enviar notificaciones y procesar pagos. En **desarrollo** no queremos enviar emails reales ni cobrar dinero real. En **producción** sí queremos hacerlo. El problema: `OrderService` no puede tener un `if ENV == 'development'` dentro; eso mezcla responsabilidades.

La solución: **una clase Factory** que centraliza esa decisión. El Service solo pregunta “dame un notificador” y la Factory le da el correcto según el entorno. *El Service nunca sabe si está usando un mock o uno real.*

El árbol de clases completo

El patrón Factory en este proyecto tiene **tres niveles**:



- **Nivel 1 — Base:** Define el *contrato* (qué métodos deben existir).
- **Nivel 2 — Implementaciones:** Dos versiones: Mock (desarrollo) y Real (producción).
- **Nivel 3 — Factory:** Decide cuál instanciar según `settings.ENV_TYPE`.

Nivel 1 — Las Clases Base (el contrato)

Las clases base definen **qué operaciones** deben hacer los notificadores y procesadores, pero **no las implementan**. Cualquier subclase *está obligada* a implementar esos métodos.

Listing 9: orders/infra/notifiers.py – BaseNotifier

```

1 class BaseNotifier:
2     """
3     Interfaz base para todos los notificadores (LSP).
4     Cualquier subclase es intercambiable donde se espere un
5     BaseNotifier.
6     """
7     def notify(self, user_email: str, message: str) -> dict:
8         raise NotImplementedError("Las subclases deben implementar
9         notify()")

```

Listing 10: orders/infra/payment.py – BasePaymentProcessor

```

1 class BasePaymentProcessor:
2     """
3     Interfaz base para procesadores de pago (LSP).
4     Cualquier subclase es intercambiable donde se espere un
5     BasePaymentProcessor.
6     """
7     def process_payment(self, amount: float) -> dict:
8         raise NotImplementedError("Las subclases deben implementar
9         process_payment()")

```

¿Por qué raise NotImplementedError?

Si alguien crea una subclase y *olvida* implementar el método, Python lanzará `NotImplementedError` en tiempo de ejecución. Es como un contrato: “si heredas de mí, **tienes** que implementar este método o tu código explota”. Esto garantiza el principio **LSP** (Liskov): todo notificador puede usarse donde se espera un `BaseNotifier`.

Nivel 2 — Las Implementaciones (Mock vs Real)

Listing 11: orders/infra/notifiers.py – MockNotifier y EmailNotifier

```

1 class MockNotifier(BaseNotifier):
2     # Usado en desarrollo: solo imprime en consola, no envia nada
3     # real
4     def notify(self, user_email: str, message: str) -> dict:
5         print(f"[MOCK NOTIFICATION] To: {user_email}")
6         print(f"[MOCK MESSAGE] {message}")
7         return {
8             'success': True,
9             'method': 'mock',
10            'recipient': user_email,
11            'message': 'Notification simulated (development mode)'
12        }
13

```

```

14 class EmailNotifier(BaseNotifier):
15     # Usado en produccion: aqui iria el codigo real de envio de
        email
16     def notify(self, user_email: str, message: str) -> dict:
17         print(f"[REAL EMAIL] Sending to: {user_email}")
18         print(f"[EMAIL CONTENT] {message}")
19         return {
20             'success': True,
21             'method': 'email',
22             'recipient': user_email,
23             'message': 'Email sent successfully'
24         }

```

Listing 12: orders/infra/payment.py – MockPaymentProcessor y RealPaymentProcessor

```

1 class MockPaymentProcessor(BasePaymentProcessor):
2     # Genera un ID de transaccion falso, no cobra nada
3     def process_payment(self, amount: float) -> dict:
4         transaction_id = f"MOCK-{random.randint(100000, 999999)}"
5         print(f"[MOCK PAYMENT] Processing ${amount}")
6         print(f"[MOCK PAYMENT] Transaction ID: {transaction_id}")
7         return {
8             'success': True,
9             'transaction_id': transaction_id,
10            'amount': amount,
11            'processor': 'mock',
12            'message': 'Payment simulated successfully (
                development mode)'
13        }
14
15
16 class RealPaymentProcessor(BasePaymentProcessor):
17     # En produccion aqui iria la integracion real con Stripe /
        PayPal / etc.
18     def process_payment(self, amount: float) -> dict:
19         print(f"[REAL PAYMENT] Processing ${amount} through payment
            gateway")
20         transaction_id = f"REAL-{random.randint(100000, 999999)}"
21         return {
22             'success': True,
23             'transaction_id': transaction_id,
24             'amount': amount,
25             'processor': 'real',
26             'message': 'Payment processed successfully'
27         }

```

¿Cuál es la diferencia real entre Mock y Real?

- **MockPaymentProcessor:** genera un ID aleatorio con prefijo MOCK-. No llama a ningún banco. Sirve para probar el flujo sin perder dinero.
- **RealPaymentProcessor:** en un sistema real, aquí iría la llamada a la API de

Stripe/PayPal/MercadoPago. Genera ID con prefijo REAL-.
El `OrderService` recibe el resultado (un dict con `success` y `transaction_id`) sin importarle quién lo generó.

Nivel 3 — Las Factories (la decisión centralizada)

Listing 13: `orders/infra/factories.py` – Las dos factories completas

```

1 from django.conf import settings
2 from .notifiers import MockNotifier, EmailNotifier
3 from .payment import MockPaymentProcessor, RealPaymentProcessor
4
5
6 class NotificationFactory:
7
8     @staticmethod
9     def create():
10         # Lee settings.ENV_TYPE = 'development' o 'production'
11         if settings.ENV_TYPE == 'development':
12             return MockNotifier()      # <-- dev: imprime en consola
13         return EmailNotifier()         # <-- prod: envia email real
14
15
16 class PaymentProcessorFactory:
17
18     @staticmethod
19     def create():
20         if settings.ENV_TYPE == 'development':
21             return MockPaymentProcessor() # <-- dev: simula pago
22         return RealPaymentProcessor()     # <-- prod: cobra de
                                         verdad

```

¿Qué hace exactamente `@staticmethod`?

Un `@staticmethod` es un método que **no depende de instancia** (`self`) ni de la clase (`cls`). Se puede llamar directamente como `NotificationFactory.create()` sin necesidad de crear un objeto `NotificationFactory()`. Es la forma correcta de un método utilitario que solo organiza lógica de creación.

¿Cómo usa `OrderService` las Factories?

Listing 14: `orders/services.py` – `OrderService` usa las Factories

```

1 class OrderService:
2
3     def __init__(self, notifier=None, payment_processor=None):
4         # Si alguien inyecta una dependencia desde afuera (tests),
           se usa esa.

```

```

5      # Si no se inyecta nada, la Factory decide cual usar.
6      self.notifier = notifier or NotificationFactory.create()
7      #
8      #                               Retorna MockNotifier o
          EmailNotifier
9      #                               segun ENV_TYPE.
          OrderService NO sabe cual.

10
11     self.payment_processor = payment_processor or
          PaymentProcessorFactory.create()
12     #
          ~~~~~
13     #                               Retorna Mock
          o Real.

14
15     def create_order(self, ...):
16         # ... mucho codigo ...

17
18         # Llamada al procesador de pago (no importa si es Mock o
          Real)
19         self.payment_processor.process_payment(float(order.total))
20
21         # Llamada al notificador (no importa si es Mock o Email)
22         self.notifier.notify(
23             user_email=customer.email,
24             message=f"Tu orden #{order.pk} fue creada. Total: ${
                order.total}"
25         )

```

El Factory vs el Builder: ¿cuál es la diferencia?

- **Builder:** construye un *objeto de dominio* (Order, Product) paso a paso, con validaciones y cálculos acumulados.
- **Factory:** decide *qué implementación* de una dependencia usar, en una sola llamada (`create()`). No construye nada complejo, solo elige e instancia.

En resumen: el Builder *ensambla*, el Factory *elige*.

Open/Closed Principle en acción

Si mañana se quiere agregar notificaciones por SMS, el proceso es:

1. Crear `SMSNotifier(BaseNotifier)` con su propio `notify()`.
2. Ajustar `NotificationFactory.create()` para retornar `SMSNotifier()`.
3. **Nada más.** `OrderService`, las vistas, los serializers — nada cambia.

Eso es *abierto para extensión*, *cerrado para modificación* (OCP).

Capa de Aplicación: los Services

Los Services son el **cerebro del sistema**. Reciben datos del mundo exterior, orquestan Builders y Factories, y devuelven resultados. No saben nada de HTTP.

OrderService

Listing 15: orders/services.py – Constructor con inyección de dependencias

```
1 class StockError(ValueError):
2     """Excepcion especifica: la vista la captura y devuelve HTTP
3     409."""
4     pass
5
6 class OrderService:
7     def __init__(self, notifier=None, payment_processor=None):
8         # Si alguien inyecta un notifier (ej: en tests), se usa ese
9         # Si no, la Factory decide cual usar.
10        self.notifier = notifier or NotificationFactory.create()
11        self.payment_processor = payment_processor or PaymentProcessorFactory.create()
12        self.customer_service = CustomerService()
13        self.product_service = ProductService()
```

Dependency Injection (Principio DIP)

El constructor `__init__(self, notifier=None)` acepta dependencias **desde afuera**. Si alguien pasa un notificador, se usa ese. Si no, la Factory lo crea. Esto permite en los *tests* hacer:

```
service = OrderService(notifier=MockNotifier())
```

Sin necesidad de modificar la clase para testearla.

Listing 16: orders/services.py – create_order (flujo completo)

```
1 def create_order(self, customer_data, items, shipping_address,
2     discount_code=None) -> dict:
3     try:
4         # Paso 1: validar que vienen los datos minimos
5         if not customer_data or not items or not shipping_address:
6             raise ValueError("Faltan datos obligatorios")
7
8         # Paso 2: obtener o crear el cliente
9         customer = self.customer_service.get_or_create_customer(
10             email=customer_data['email'],
11             data=customer_data,
12         )
13
14         # Paso 3: iniciar el Builder
```

```
15     builder = (  
16         OrderBuilder()  
17         .for_customer(customer)  
18         .with_shipping_address(shipping_address)  
19     )  
20  
21     # Paso 4: resolver cada variante y añadirla al builder  
22     for item in items:  
23         variant = ProductVariant.objects.select_related(  
24             'product', 'inventory'  
25         ).get(pk=item['variant_id'])  
26  
27         try:  
28             builder.add_item(variant=variant, quantity=item['  
29                 quantity'])  
30         except ValueError as stock_err:  
31             raise StockError(str(stock_err)) # convierte a  
32             StockError  
33  
34     if discount_code:  
35         builder.with_discount(discount_code)  
36  
37     # Paso 5: construir y guardar la orden (llama a build())  
38     order = builder.build()  
39  
40     # Paso 6: descontar el stock del inventario  
41     for item in items:  
42         self.product_service.reserve_stock(  
43             variant_id=item['variant_id'],  
44             quantity=item['quantity'],  
45         )  
46  
47     # Paso 7: procesar pago (Factory decide Mock vs Real)  
48     self.payment_processor.process_payment(float(order.total))  
49  
50     # Paso 8: notificar al cliente (no critico: no detiene el  
51     flujo)  
52     try:  
53         self.notifier.notify(  
54             user_email=customer.email,  
55             message=f"Tu orden #{order.pk} fue creada. Total: $  
56                 {order.total}"  
57         )  
58     except Exception as e:  
59         print(f"[Warning] Notificacion fallida: {e}")  
60  
61     return {  
62         'success': True,  
63         'order_id': order.pk,  
64         'total': float(order.total),  
65         'message': 'Orden creada exitosamente',  
66     }
```

```

62     }
63
64     except StockError as e:
65         return {'success': False, 'error_type': 'stock', 'message':
66                 str(e)}
67     except ValueError as e:
68         return {'success': False, 'message': str(e)}
69     except Exception as e:
70         return {'success': False, 'message': f'Error: {str(e)}'}

```

Por qué StockError es una subclase de ValueError

`StockError(ValueError)` hereda de `ValueError`. Esto permite al `except StockError` ser más específico que el `except ValueError` general. El *Service* sabe distinguir “falló por stock” de “falló por datos inválidos”, y la *Vista* puede devolver HTTP 409 solo cuando es un error de stock.

ProductService

Listing 17: products/services.py – Métodos principales

```

1 class ProductService:
2
3     def create_product(self, data: dict) -> Product:
4         """Delega la creacion al ProductBuilder."""
5         builder = ProductBuilder()
6         builder.with_name(data['name'])
7         builder.with_description(data.get('description', ''))
8         builder.with_category(data.get('category', 'other'))
9         for v in data.get('variants', []):
10             builder.add_variant(sku=v['sku'], price=v['price'],
11                                initial_stock=v.get('initial_stock',
12                                                    0),
13                                size=v.get('size'), color=v.get('
14                                    color'))
15
16         return builder.build()
17
18     def reserve_stock(self, variant_id: int, quantity: int) -> dict:
19         """Descuenta stock. Retorna dict con éxito/fracaso."""
20         variant = ProductVariant.objects.select_related('inventory'
21                                                         ).get(id=variant_id)
22         inventory = variant.inventory
23         if not inventory.has_stock(quantity):
24             return {'success': False, 'message': 'Stock
25                 insuficiente'}
26         inventory.available_quantity -= quantity
27         inventory.save()
28         return {'success': True,
29                 'message': f'Restante: {inventory.
30                     available_quantity}'}

```

```

25
26     def update_stock(self, variant_id: int, quantity: int) -> dict:
27         """Actualiza el stock absoluto de una variante."""
28         inventory = Inventory.objects.get(product_variant__id=
                variant_id)
29         inventory.available_quantity = quantity
30         inventory.save()
31         return {'success': True, 'new_stock': inventory.
                available_quantity,
32                 'is_below_minimum': inventory.is_below_minimum()}

```

CustomerService

Listing 18: customers/services.py – get_or_create_customer

```

1 class CustomerService:
2
3     def get_or_create_customer(self, email: str, data: dict) ->
4         Customer:
5         """
6         Si ya existe un cliente con ese email: lo devuelve (y
7         actualiza si cambio).
8         Si no existe: lo crea.
9         """
10        customer, created = Customer.objects.get_or_create(
11            email=email,
12            defaults={
13                'nombre': data.get('nombre', ''),
14                'apellido': data.get('apellido', ''),
15                'telefono': data.get('telefono', ''),
16                'direccion': data.get('direccion', ''),
17            }
18        )
19        if not created: # ya existia: actualizar campos si
20            cambiaron
21            for field in ('nombre', 'apellido', 'telefono', '
22                direccion'):
23                if data.get(field):
24                    setattr(customer, field, data[field])
25            customer.save()
26        return customer
27
28     def create_customer(self, data: dict) -> dict:
29         """Crea un cliente nuevo. La unicidad de email es logica de
30         negocio:
31         vive aqui, NO en el serializer."""
32         if Customer.objects.filter(email=data.get('email')).exists
33             ():
34             return {'success': False, 'customer': None,

```

```

29         'message': f"Email {data['email']} ya
           registrado"}
30     customer = Customer.objects.create(**data)
31     return {'success': True, 'customer': customer,
32           'message': 'Cliente creado exitosamente'}

```

Lógica de negocio en Serializers: ¡PROHIBIDO!

La unicidad del email es una **regla de negocio**: “no pueden existir dos clientes con el mismo email”. Por eso vive en `CustomerService`, no en `CreateCustomerSerializer`. Los serializers solo validan **formato** (¿es un email válido?, ¿el texto no está vacío?), nunca consultan la base de datos.

Capa de Presentación: Views y Serializers

Serializers: validación de entrada/salida

Listing 19: orders/serializers.py – CheckoutSerializer (entrada)

```

1 class CheckoutItemSerializer(serializers.Serializer):
2     variant_id = serializers.IntegerField() # ID de la
        variante
3     quantity = serializers.IntegerField(min_value=1)
4     # product_name y price son opcionales: el backend los genera
        como snapshot
5     product_name = serializers.CharField(required=False,
        allow_blank=True)
6     price = serializers.DecimalField(max_digits=10,
        decimal_places=2,
7                                     required=False)
8
9 class CheckoutSerializer(serializers.Serializer):
10    customer = CheckoutCustomerSerializer()
11    items = CheckoutItemSerializer(many=True)
12    shipping_address = serializers.CharField()
13    discount_code = serializers.CharField(required=False,
        allow_null=True)
14
15    def validate_items(self, items):
16        if not items:
17            raise serializers.ValidationError(
18                "La orden debe tener al menos un item."
19            )
20        return items

```

Interface Segregation Principle en los Serializers

En lugar de un solo serializer “*Hacelo Todo*”, hay serializers específicos:

- `OrderSerializer` — solo para **leer** órdenes (GET)
- `CheckoutSerializer` — solo para **crear** órdenes (POST checkout)
- `CreateProductSerializer` — solo para **crear** productos
- `UpdateProductSerializer` — solo para **actualizar** (PATCH)
- `UpdateStockSerializer` — solo para actualizar el stock

Cada serializer tiene un propósito único. Eso es **ISP**.

Views: solo HTTP, sin lógica

Listing 20: orders/views.py – CreateOrderView (checkout)

```

1 class CreateOrderView(APIView):
2     def post(self, request):
3         # 1. Validar el formato de los datos (serializer)
4         serializer = CheckoutSerializer(data=request.data)
5         if not serializer.is_valid():
6             return Response({'errors': serializer.errors},
7                             status=status.HTTP_400_BAD_REQUEST)
8
9         # 2. Delegar toda la logica al service
10        data = serializer.validated_data
11        result = OrderService().create_order(
12            customer_data = data['customer'],
13            items = data['items'],
14            shipping_address = data['shipping_address'],
15            discount_code = data.get('discount_code'),
16        )
17
18        # 3. Traducir el resultado del service a un codigo HTTP
19        if result['success']:
20            return Response(result, status=status.HTTP_201_CREATED)
21        if result.get('error_type') == 'stock':
22            return Response(result, status=status.HTTP_409_CONFLICT)
23        return Response(result, status=status.HTTP_400_BAD_REQUEST)

```

Listing 21: orders/views.py – AdminDashboardView

```

1 class AdminDashboardView(APIView):
2     def get(self, request):
3         # Una sola linea: delega TODO al service
4         stats = OrderService().get_dashboard_stats()
5         return Response(stats)

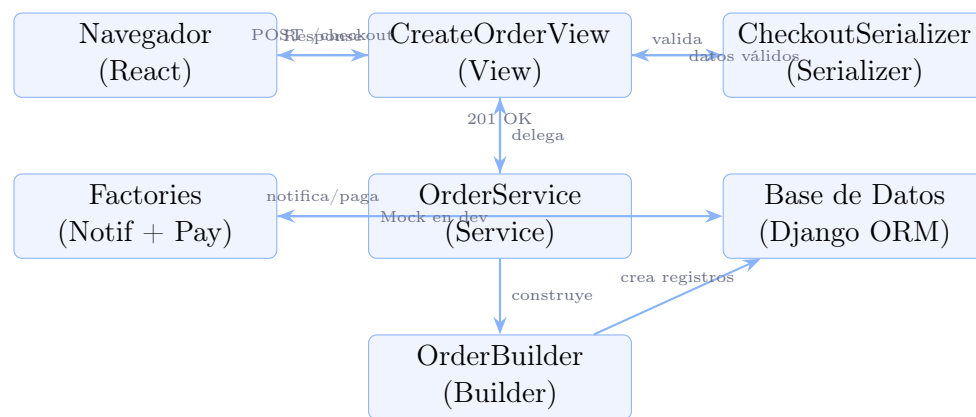
```

Resumen de los Principios SOLID en el Proyecto

Principio	Dónde se aplica	Evidencia concreta
S — SRP	Todas las capas	Views $\neq$ Services $\neq$ Builders $\neq$ Factories
O — OCP	NotificationFactory	Agregar SMSNotifier sin tocar OrderService
L — LSP	BaseNotifier, BasePaymentProcessor	Mock y Real son intercambiables
I — ISP	Serializers	Create/Update/Read separados
D — DIP	OrderService.__init__	notifier=None acepta inyección desde fuera

Flujo Completo de una Compra (Checkout)

Este diagrama muestra el camino que recorre una petición desde el navegador hasta la base de datos:



Paso a paso:

1. El **navegador** envía **POST /api/orders/checkout/** con datos JSON.
2. La **vista** pasa los datos al **CheckoutSerializer**.
3. El **serializer** valida formato (campos obligatorios, tipos). Si hay error $\rightarrow$ HTTP 400.
4. La **vista** delega al **OrderService.create_order()**.
5. El **service** llama a **CustomerService** para obtener/crear el cliente.
6. El **service** inicia el **OrderBuilder** y le agrega variantes una a una.
7. El **builder** valida el stock en cada **add_item()**.
8. El **builder** calcula totales y llama a **Order.objects.create()**.
9. El **service** llama a **ProductService.reserve_stock()** para cada ítem.
10. El **service** llama a **PaymentProcessorFactory.create().process_payment()**.
11. El **service** llama a **NotificationFactory.create().notify()**.

12. La **vista** devuelve HTTP 201 con el ID y total de la orden.

Estructura de Carpetas y su Justificación

```
CEFUSAECommerce/  
  products/          # App independiente: Product, Variant, Inventory  
    api/  
      views.py  
      serializers.py  
    models.py  
    services.py      # ProductService  
    urls.py  
    admin.py  
    apps.py  
    tests.py  
    domain/  
      builders.py    # ProductBuilder  
  
  customers/         # App independiente: Customer  
    api/  
      views.py  
      serializers.py  
    models.py  
    services.py      # CustomerService  
    urls.py  
    admin.py  
    apps.py  
    tests.py  
  
  orders/            # App independiente: Order, OrderItem  
    api/  
      views.py  
      serializers.py  
    models.py  
    services.py      # OrderService (orquesta todo)  
    urls.py  
    admin.py  
    apps.py  
    tests.py  
    domain/  
      builders.py    # OrderBuilder (patron Builder)  
  infra/  
    factories.py     # NotificationFactory, PaymentProcessorFactory  
    notifiers.py     # BaseNotifier, MockNotifier, EmailNotifier  
    payment.py       # BasePaymentProcessor, Mock, Real
```

¿Por qué domain/ e infra/ son subcarpetas de orders/?

- **domain/** — contiene lógica *pura de negocio* (Builders). No depende de HTTP ni de servicios externos.
- **infra/** — contiene *detalles técnicos* (Factories, notificadores, pasarelas de pago). Pueden cambiar sin afectar la lógica de negocio.

Esta separación es la base de la **Arquitectura Limpia** (Clean Architecture).

Endpoints Disponibles

Método + URL	Descripción	HTTP
POST /api/orders/checkout/	Crear una orden (checkout)	201 / 400 / 409
GET /api/orders/<id>/	Detalle de una orden	200 / 404
PATCH /api/orders/<id>/status/	Cambiar estado	200 / 400
GET /api/products/	Listar productos activos	200
POST /api/products/	Crear producto	201 / 400
GET /api/products/<id>/	Detalle de producto	200 / 404
PATCH /api/products/<id>/	Actualizar producto	200 / 404
DELETE /api/products/<id>/	Desactivar producto	204 / 404
POST /api/products/check-stock/	Verificar stock	200
GET /api/customers/	Listar clientes	200
POST /api/customers/	Crear cliente	201 / 409
GET /api/customers/<id>/	Detalle de cliente	200 / 404
GET /api/customers/<id>/orders/	Órdenes del cliente	200 / 404
GET /api/admin/dashboard/	Estadísticas admin	200
GET /api/admin/orders/	Listar órdenes (admin)	200
DELETE /api/admin/orders/<id>/	Eliminar orden	204 / 404